

DOCUMENTACIÓN TÉCNICA DEL SISTEMA

SISTEMA DE GESTIÓN EMPRESARIAL

CONSORCIOS VILLEGAS E.I.R.L.

Manual técnico y guía para desarrolladores



Logotipo institucional de Consorcios Villegas E.I.R.L.

Código del documento: VT-DEV-001

Repositorio: <https://github.com/JhonMM27/Villegas2026>

Rama revisada: main

Commit de referencia: 7b99815c8778

Fecha de revisión: 29/08/2026

Versión del documento: 1.1

Control del documento

Campo	Valor
Documento	Documentación Técnica / Manual del Desarrollador
Código	VT-DEV-001
Sistema	Villegas2026 - Consorcios Villegas E.I.R.L.
Repositorio	https://github.com/JhonMM27/Villegas2026
Rama	main
Commit de referencia	7b99815c87786c3b5e13be5b2890a77d4b2aa9c8
Fecha de revisión	30/08/2026
Versión	1.2
Responsable	Consorcios Villegas E.I.R.L. - Administración / TI
Aprobado por	Propietario del sistema
Público objetivo	Desarrolladores, mantenedores, responsables de TI, soporte y responsables técnicos.

Historial de versiones

Versión	1.2	Responsable	Descripción
1.0	29/08/2026	Consorcios Villegas E.I.R.L.	Primera edición separada y completa, basada en revisión del repositorio Villegas2026.

Criterio de evidencia

Para no confundir funcionalidades implementadas con propuestas, el documento distingue tres niveles de evidencia:

Etiqueta	Significado
VERIFICADO	Existe evidencia directa en código, rutas, modelos, servicios, pruebas o documentos del repositorio.
INFERIDO	Conclusión obtenida de la relación entre varios elementos del código; debe confirmarse en el ambiente real cuando corresponda.
RECOMENDADO	Buena práctica de operación, seguridad o despliegue. No implica que esté configurada actualmente.
Seguridad documental: no incluir contraseñas, APP_KEY, tokens, certificados privados, credenciales de base de datos ni valores completos del archivo .env en capturas o anexos.	

Fuentes técnicas revisadas

ID	Fuente	Uso
R1	README.md	Identidad general del proyecto.

ID	Fuente	Uso
R2	AGENTS.md	Stack, convenciones, servicios, reglas de kardex e inmutabilidad.
R3	routes/web.php	Mapa funcional, rutas y módulos.
R4	composer.json	Dependencias PHP y scripts Composer.
R5	package.json	Dependencias frontend y scripts Vite.
R6	app/Models	Entidades y relaciones Eloquent.
R7	app/Services	Lógica de negocio y servicios de dominio.
R8	database/migrations	Estructura y evolución del esquema.
R9	database/seeder/RolesAndPermissionsSeeder.php	Roles, permisos y provisión inicial.
R10	tests/Feature y tests/Unit	Cobertura de pruebas existente; no se presume ejecución exitosa.
R11	PLANILLA.md	Funcionamiento documentado del módulo de planilla.

Referencias externas usadas únicamente para recomendaciones de documentación/despliegue:

Referencia	Aplicación
ISO/IEC/IEEE 15289:2019	Estructura de información del ciclo de vida de sistemas y software.
ISO/IEC/IEEE 26514:2022	Diseño y desarrollo de información para usuarios de software.
Laravel 12 Documentation	Buenas prácticas de instalación, configuración y despliegue.
C4 Model	Convención recomendada para diagramas de arquitectura.

Índice del documento

Índice materializado sin número de página porque la paginación cambiará al reemplazar los recuadros por capturas. Al finalizar, en Microsoft Word inserte o actualice una Tabla de contenido automática.

Sección	Contenido
1	Resumen ejecutivo
2	Introducción
3	Objetivos
4	Alcance del sistema
5	Actores, roles y autorización
6	Requisitos funcionales
7	Requisitos no funcionales
8	Reglas de negocio

Sección	Contenido
9	Arquitectura del software
10	Diseño técnico y tecnologías
11	Modelo de datos
12	Rutas e interfaces funcionales
13	Seguridad
14	Configuración del sistema
15	Instalación para desarrollo
16	Despliegue a producción
17	Flujos críticos
18	Reportes y exportaciones
19	Auditoría y trazabilidad
20	Pruebas y validación
21	Respaldo y recuperación
22	Mantenimiento y operaciones
23	Limitaciones y deuda técnica
24	Conclusiones y recomendaciones
Anexo A	Glosario
Anexo B	Referencia rápida de comandos
Anexo C	Patrón de permisos
Anexo D	Checklist de diagramas e imágenes técnicas
Anexo E	Referencias y pendientes de validación

1. Resumen ejecutivo

El repositorio Villegas2026 implementa un sistema web de gestión empresarial para Consorcios Villegas E.I.R.L. desarrollado con Laravel 12, PHP 8.2 o superior, MySQL y una interfaz basada en Blade, Tailwind CSS y Vite. La aplicación centraliza operaciones comerciales y administrativas: maestros, productos, clientes, proveedores, compras, ventas, cotizaciones, movimientos de inventario y kardex, producción/preparadas, núcleos, caja, gastos, costos, cuentas corrientes, rentabilidad, préstamos, planilla, reportes y auditoría. [R1][R2][R3]

La arquitectura observable es un monolito modular de Laravel con una separación explícita entre controladores HTTP, modelos Eloquent y una capa de servicios de negocio. El inventario utiliza MovimientoService como punto central para registrar entradas y salidas y conservar la cadena del kardex valorizado mediante Costo Promedio Ponderado (CPP). Las transacciones críticas de compras, ventas y preparados se manejan como registros inmutables después de su creación y, cuando requieren corrección, utilizan flujos de anulación y rectificación. [R2]

La revisión también identificó mecanismos relevantes de seguridad y trazabilidad: autenticación, usuarios activos/inactivos, control de acceso por roles y permisos con Spatie, limitación de intentos de inicio de sesión, auditoría administrativa y pruebas enfocadas en kardex, rectificaciones, cuentas corrientes, planilla y auditoría. [R7][R8][R9][R10]

Hallazgo prioritario: El repositorio contiene un archivo .env versionado y el seeder de roles crea una cuenta administrativa de prueba con credenciales definidas en código. Este documento no expone esos valores. Antes de un uso productivo se recomienda retirar secretos del control de versiones, rotar credenciales y reemplazar cualquier cuenta de prueba por un proceso seguro de aprovisionamiento. [R8][W4][W7]

2. Introducción

2.1. Contexto

El sistema agrupa en una sola aplicación procesos que normalmente se distribuyen entre ventas, compras, inventario, caja, costos, cuentas por cobrar/pagar y gestión de personal. La existencia de módulos conectados por entidades comunes -productos, clientes, proveedores, movimientos, empleados y documentos- permite mantener trazabilidad operativa y generar reportes consolidados.

El repositorio no contiene una declaración formal única del “problema de negocio” ni un documento de requisitos original. Por ello, esta documentación no atribuye necesidades organizacionales no demostradas. El propósito se describe a partir de las funcionalidades realmente presentes en código y rutas. [R1][R3]

2.2. Propósito de esta documentación

- Describir la arquitectura, componentes, tecnologías y reglas de negocio del sistema.
- Registrar los módulos y rutas funcionales observadas en la rama main.
- Explicar el modelo de autorización, las reglas de kardex y las particularidades de planilla.
- Proporcionar una guía de instalación, operación, pruebas, mantenimiento, respaldo y despliegue.
- Incluir un manual de usuario orientado a tareas, con marcadores exactos para incorporar capturas de pantalla posteriormente.
- Separar claramente los elementos verificados en el repositorio de las recomendaciones de mejora.

2.3. Público objetivo

Perfil	Necesidad principal
Desarrollador / mantenedor	Comprender arquitectura, módulos, servicios, modelos, rutas y reglas de negocio.
Administrador del sistema	Configurar usuarios, roles, catálogos, operación, reportes y mantenimiento.
Soporte / TI	Instalar, diagnosticar, respaldar, restaurar y desplegar el sistema.
Usuario operativo	Utilizar compras, ventas, inventario, caja, costos y otros módulos autorizados.
Responsable de planilla	Gestionar empleados, adelantos, préstamos, pagos, inasistencias, vacaciones y reportes.
Auditor / supervisor	Revisar trazabilidad, reportes, rectificaciones y eventos de auditoría.

3. Objetivos

3.1. Objetivo general

INFERIDO - Centralizar y controlar los principales procesos comerciales, logísticos, productivos, financieros y de personal de Consorcios Villegas E.I.R.L. mediante una aplicación web integrada que mantenga trazabilidad de operaciones, inventario y reportes para la toma de decisiones.

3.2. Objetivos específicos

- Administrar catálogos maestros, usuarios, roles y permisos.
- Gestionar productos, clientes y proveedores como entidades centrales del negocio.
- Registrar compras, ventas, cotizaciones y operaciones provisionales con trazabilidad documental.
- Mantener el inventario mediante un kardex valorizado y una cadena cronológica de movimientos.
- Gestionar procesos de preparación/producción, núcleos y formulaciones.
- Registrar movimientos de caja, gastos, costos, préstamos y cuentas corrientes.
- Calcular y controlar componentes de planilla, inasistencias, vacaciones, adelantos y préstamos a empleados.
- Generar reportes operativos en pantalla y, según el módulo, exportaciones PDF, Excel y archivos ZIP.
- Mantener control de acceso mediante roles/permisos y registrar eventos de auditoría.

4. Alcance del sistema

4.1. Funcionalidades incluidas

Área	Alcance verificado
Seguridad	Inicio/cierre de sesión, usuario activo, perfil, usuarios, roles y permisos.
Maestros	Unidades, tipos de afectación, documentos, comprobantes, cobranza, formas/medios de pago, líneas, operaciones, series y configuraciones.
Terceros	Clientes y proveedores, búsquedas y reportes asociados.
Productos e inventario	Productos, fracciones, stock, stock mínimo y kardex valorizado.
Compras	Registro, consulta, impresión, anulación, compras provisionales y reportes.
Ventas	Registro, consulta, impresión, anulación, rectificación, ventas provisionales, entregas y reportes.
Cotizaciones	CRUD de cotizaciones y emisión/impresión.
Producción y formulación	Formulaciones, fórmulas de alimento, fórmulas para cerdo, preparadas, núcleos y núcleos preparados.

Área	Alcance verificado
Caja	Ingresos, pagos y reportes de caja.
Gastos y costos	Registro por categorías/tipos, reportes generales y detallados.
Finanzas comerciales	Cuentas corrientes de clientes/proveedores, créditos, estados de cuenta y rentabilidad.
Préstamos	Préstamos generales con impresión, anulación y rectificación.
Planilla	Empleados, sueldos, adelantos, préstamos, pagos, inasistencias, vacaciones y reportes.
Auditoría	Consulta de eventos de auditoría restringida al perfil administrador.

4.2. Límites y elementos no evidenciados

- No se observó una API REST pública versionada como producto independiente; las rutas revisadas son principalmente rutas web de Laravel y endpoints AJAX internos. [R3]
- No se observó una aplicación móvil nativa ni un cliente de escritorio separado.
- No se encontró docker-compose.yml ni Dockerfile en la rama revisada; el despliegue con contenedores no forma parte de la implementación documentada.
- No se encontró un flujo de GitHub Actions bajo .github/workflows en la rama revisada; la integración/entrega continua no está documentada como parte del repositorio.
- Aunque existen catálogos y certificados relacionados con SUNAT, esta revisión no encontró evidencia suficiente para afirmar un flujo completo de facturación electrónica hacia SUNAT; por tanto se documenta únicamente la configuración observada, no una integración tributaria completa.
- La versión del software no aparece declarada explícitamente; debe definirse una política de versionado para releases.
- No existe un archivo .env.example en la rama revisada; la configuración reproducible de ambientes necesita formalización.

5. Actores, roles y autorización

El sistema usa spatie/laravel-permission. El modelo User incluye el trait HasRoles y los permisos se registran en base de datos. [R7][R8][W5]

Rol	Asignación verificada	Consideraciones
admin	Recibe todos los permisos CRUD generados y permisos de reportes/administración adicionales.	Es el rol con mayor alcance. La ruta de auditoría además exige rol administrator/admin según controlador/ruta; validar nombre exacto usado en producción.
vendedor	Permisos de lista/creación/edición/eliminación para ventas según seeder.	Las acciones efectivas dependen también de las rutas/controladores; las ventas se manejan como transacciones inmutables y se corrigen por anulación/rectificación.
operador	Permisos *_list y *_create de las tablas enumeradas por el seeder.	Puede consultar y crear en múltiples módulos, pero no obtiene por defecto edición/eliminación.

Adicionalmente, el acceso requiere que el usuario autenticado esté marcado como activo. Un usuario con credenciales válidas pero estado inactivo no puede continuar la sesión. [R9]

Nota de seguridad: La matriz de permisos debe administrarse bajo mínimo privilegio. Los permisos generados en el seeder son una base técnica; antes de producción conviene validar cada rol contra funciones reales del puesto.

6. Requisitos funcionales consolidados

ID	Módulo	Requisito	Estado
RF-001	Autenticación	Permitir iniciar sesión mediante correo y contraseña.	VERIFICADO
RF-002	Autenticación	Bloquear el acceso a usuarios inactivos.	VERIFICADO
RF-003	Autenticación	Limitar intentos reiterados de inicio de sesión.	VERIFICADO
RF-004	Perfil	Permitir consultar y actualizar datos de perfil autorizados.	VERIFICADO
RF-005	Seguridad	Administrar usuarios, roles y permisos.	VERIFICADO
RF-006	Dashboard	Mostrar ventas del día, compras del día, total de clientes y total de productos.	VERIFICADO
RF-007	Dashboard	Mostrar alertas de productos en o debajo del stock mínimo y productos con stock negativo.	VERIFICADO
RF-008	Maestros	Gestionar unidades, tipos de afectación, documentos, comprobantes, cobranza, pagos, líneas y tipos de operación.	VERIFICADO
RF-009	Configuración	Gestionar series de comprobantes, certificados SUNAT y configuraciones internas.	VERIFICADO
RF-010	Productos	Registrar, consultar, modificar y administrar productos y fracciones.	VERIFICADO
RF-011	Clientes	Registrar, consultar, modificar y buscar clientes.	VERIFICADO
RF-012	Proveedores	Registrar, consultar, modificar y buscar proveedores.	VERIFICADO
RF-013	Compras	Registrar compras con sus detalles y consultar/impimir la operación.	VERIFICADO
RF-014	Compras	Permitir anular compras; las transacciones no se editan/eliminan como CRUD convencional.	VERIFICADO
RF-015	Ventas	Registrar ventas con detalles y consultar/imprimir la operación.	VERIFICADO
RF-016	Ventas	Permitir anular y rectificar ventas.	VERIFICADO
RF-017	Cotizaciones	Gestionar cotizaciones y permitir su impresión.	VERIFICADO
RF-018	Provisionales	Gestionar compras y ventas provisionales y consultar saldos pendientes.	VERIFICADO
RF-019	Entregas	Gestionar entregas de ventas, pendientes, anulación y rectificación.	VERIFICADO
RF-020	Caja	Registrar ingresos de caja y consultar reportes general y detallado; Caja - Pagos no está habilitado en el menú activo.	VERIFICADO
RF-021	Préstamos	Registrar, consultar, imprimir, anular y rectificar préstamos generales.	VERIFICADO
RF-022	Producción	Gestionar formulaciones con sus detalles.	VERIFICADO
RF-023	Nutrición	Gestionar fórmulas de alimento y fórmulas de alimento para cerdo, con exportaciones.	VERIFICADO
RF-024	Producción	Gestionar preparadas, núcleos y núcleos preparados.	VERIFICADO
RF-025	Inventario	Registrar cada variación de stock en movimientos de kardex.	VERIFICADO
RF-026	Inventario	Mantener stock anterior/nuevo y costo anterior/nuevo para trazabilidad del CPP.	VERIFICADO
RF-027	Inventario	Recalcular cronológicamente el kardex cuando se registran movimientos retroactivos.	VERIFICADO
RF-029	Gastos	Gestionar gastos, categorías y tipos; generar reportes.	VERIFICADO
RF-030	Costos	Gestionar costos, categorías y tipos; generar reportes.	VERIFICADO
RF-031	Cuenta corriente clientes	Consultar créditos por cobrar, saldos, estados de cuenta y reportes.	VERIFICADO
RF-032	Cuenta corriente proveedores	Consultar compras, créditos por pagar, saldos y estados de cuenta.	VERIFICADO
RF-033	Rentabilidad	Generar reportes de rentabilidad por ventas, detalle y productos.	VERIFICADO
RF-034	Planilla	Gestionar empleados y estado activo/inactivo.	VERIFICADO
RF-035	Planilla	Registrar adelantos y distribuir el importe en Principal/Depósito/Consortio.	VERIFICADO
RF-036	Planilla	Registrar préstamos a empleados, pagos parciales y saldo pendiente.	VERIFICADO
RF-037	Planilla	Generar pagos por mes/año para empleados y controlar duplicidad por periodo.	VERIFICADO
RF-038	Planilla	Calcular total a pagar con sueldo base, horas extras y descuento de faltas.	VERIFICADO
RF-039	Planilla	Registrar inasistencias y considerar media jornada como 0.5 días.	VERIFICADO
RF-040	Planilla	Gestionar vacaciones y reportes asociados.	VERIFICADO
RF-041	Reportes	Generar reportes de ventas, compras, stock, kardex, caja, planilla, gastos, costos y cuentas corrientes.	VERIFICADO

ID	Módulo	Requisito	Estado
RF-042	Exportación	Generar salidas PDF/Excel/ZIP donde los módulos lo implementan.	VERIFICADO
RF-043	Auditoría	Permitir a administradores consultar eventos de auditoría y detalle.	VERIFICADO
RF-044	Rectificación	Registrar historial/auditoría de rectificaciones en operaciones que lo soportan.	VERIFICADO

7. Requisitos no funcionales y criterios de calidad

ID	Categoría	Criterio	Estado
RNF-001	Compatibilidad técnica	PHP ^8.2 y Laravel 12.0.	VERIFICADO
RNF-002	Persistencia	MySQL como motor de base de datos.	VERIFICADO
RNF-003	Presentación	Blade + Tailwind CSS 4 + Vite para interfaz web.	VERIFICADO
RNF-004	Seguridad	Contraseñas ocultas en serialización y casteadas como hashed en User.	VERIFICADO
RNF-005	Autorización	Roles y permisos persistidos mediante Spatie Laravel Permission.	VERIFICADO
RNF-006	Integridad	Operaciones multi-paso deben usar transacciones de base de datos según guía del proyecto.	VERIFICADO COMO CONVENCIÓN
RNF-007	Trazabilidad	Todo cambio de stock de los flujos centrales debe pasar por MovimientoService.	VERIFICADO
RNF-008	Mantenibilidad	Separación Controllers / Models / Services / Exports / View Components.	VERIFICADO
RNF-009	Calidad	Laravel Pint y PHPUnit disponibles como herramientas del proyecto.	VERIFICADO
RNF-010	Producción	APP_DEBUG=false, servidor apuntando a public/index.php y optimización de configuración.	RECOMENDADO [W3]
RNF-011	Secretos	No almacenar .env ni credenciales en el repositorio.	RECOMENDADO [W4][W7]
RNF-012	Disponibilidad	Definir RPO/RT0, periodicidad de backups y procedimiento de restauración probado.	RECOMENDADO [W6]
RNF-013	Observabilidad	Centralizar logs y definir alertas para fallos de autenticación, kardex y transacciones.	RECOMENDADO
RNF-014	CI/CD	Ejecutar pruebas y análisis de estilo automáticamente en pull requests.	RECOMENDADO
RNF-015	Rendimiento	Definir SLO de respuesta y pruebas de carga para reportes/consultas de alto volumen.	RECOMENDADO

8. Reglas de negocio relevantes

ID	Regla
RN-INV-01	Todo cambio de inventario en los flujos centrales debe registrarse a través de MovimientoService para preservar la cadena del kardex.
RN-INV-02	Cada movimiento guarda stock anterior/nuevo y costo anterior/nuevo, permitiendo trazabilidad del CPP móvil.
RN-INV-03	Si una entrada o salida se registra con fecha anterior al último movimiento del producto, el sistema debe recalcular la cadena en orden fecha, id.
RN-INV-04	Las operaciones de inventario deben utilizar MovimientoService y conservar la trazabilidad del kardex.
RN-INV-05	productos.stock_almacen se maneja en sacos/empaques, no en kilogramos.
RN-INV-06	En nucleo_preparada_detalles, el campo salida_kg está documentado internamente como nombre engañoso porque en ese contexto almacena sacos.
RN-TRX-01	Compras, ventas y registros de preparados se consideran inmutables una vez creados; las correcciones usan anulación y/o rectificación.
RN-TRX-02	Las rectificaciones retroactivas deben mantener el orden cronológico del kardex.
RN-PLA-01	En adelantos, la suma Principal + Depósito + Consorcio no puede superar el monto del adelanto.
RN-PLA-02	Al crear un préstamo de planilla, saldo_pendiente inicia igual a monto_original; cada pago parcial reduce ese saldo.
RN-PLA-03	Cuando saldo_pendiente llega a cero, el préstamo pasa al estado pagado.
RN-PLA-04	El pago mensual se calcula como sueldo_base + horas_extras - descuento_faltas.
RN-PLA-05	No se debe registrar dos veces el mismo pago de mes/año para el mismo empleado.
RN-PLA-06	Una inasistencia de medio día equivale a 0.5 días para el cálculo de faltas.
RN-PLA-07	El descuento de faltas se documenta como dias_faltados * (sueldo_real / 30).

ID	Regla
RN-PLA-08	Los adelantos no se descuentan automáticamente de la planilla; deben considerarse manualmente durante el proceso de pago.
RN-SEC-01	Solo usuarios activos pueden mantener una sesión autenticada.
RN-SEC-02	La capacidad de ejecutar una acción depende de los permisos/roles y de las rutas/controladores disponibles.

9. Arquitectura del software

9.1. Estilo arquitectónico

INFERIDO - La aplicación sigue una arquitectura monolítica modular sobre Laravel. La interfaz se renderiza principalmente con Blade y recursos frontend compilados con Vite; los controladores reciben las solicitudes HTTP y delegan lógica de negocio a servicios de dominio; los modelos Eloquent representan la persistencia en MySQL. [R2][R3][R4][R5]

DIAGRAMA PENDIENTE DT-01
Diagrama de contexto del sistema
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-01. Diagrama de contexto del sistema. Fuente: código PlantUML verificado contra el proyecto.

9.2. Vista lógica de contenedores

DIAGRAMA PENDIENTE DT-02
Diagrama de arquitectura lógica / contenedores
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-02. Diagrama de arquitectura lógica / contenedores. Fuente: código PlantUML verificado contra el proyecto.

9.3. Capas y responsabilidades

Capa	Tecnología / carpeta	Responsabilidad
Presentación	resources/views + Blade + Tailwind CSS + Vite	Pantallas, formularios, tablas, modales y componentes visuales.
HTTP	app/Http/Controllers + routes/web.php	Recepción de solicitudes, validación, autorización, orquestación y respuestas.
Dominio/Servicios	app/Services	Lógica transaccional y reglas complejas: compras, ventas, kardex, planilla, reportes, etc.
Persistencia	app/Models + Eloquent	Entidades, relaciones, casts y acceso a MySQL.
Exportación	app/Exports + DOMPDF + Laravel Excel	Generación de PDF, Excel y otros formatos de salida.
Seguridad	Laravel Auth + Spatie Permission	Autenticación y autorización por roles/permisos.
Pruebas	tests/Feature + tests/Unit + scripts de verificación	Validación de controladores, servicios, rectificaciones y kardex.

9.4. Servicios de dominio detectados

Servicio	Responsabilidad general
AuditoriaService	Lógica de negocio del dominio Auditoria.
CompraProvisionalService	Lógica de negocio del dominio CompraProvisional.
CompraService	Lógica de negocio del dominio Compra.
EmpleadoService	Lógica de negocio del dominio Empleado.
EmpleadoSueldoService	Lógica de negocio del dominio EmpleadoSueldo.
FormulaAlimentoCerdoService	Lógica de negocio del dominio FormulaAlimentoCerdo.
FormulaAlimentoService	Lógica de negocio del dominio FormulaAlimento.
MovimientoService	Lógica de negocio del dominio Movimiento.
NucleoPreparadaService	Lógica de negocio del dominio NucleoPreparada.
NucleoService	Lógica de negocio del dominio Nucleo.
PlanillaAdelantoService	Lógica de negocio del dominio PlanillaAdelanto.
PlanillaAsistenciaService	Lógica de negocio del dominio PlanillaAsistencia.
PlanillaInasistenciaService	Lógica de negocio del dominio PlanillaInasistencia.
PlanillaPagoService	Lógica de negocio del dominio PlanillaPago.
PlanillaPrestamoService	Lógica de negocio del dominio PlanillaPrestamo.
PreparadaService	Lógica de negocio del dominio Preparada.
PrestamoService	Lógica de negocio del dominio Prestamo.
RectificacionAuditoriaService	Lógica de negocio del dominio RectificacionAuditoria.
ReporteCajaService	Lógica de negocio del dominio ReporteCaja.
VentaEntregaService	Lógica de negocio del dominio VentaEntrega.
VentaProvisionalService	Lógica de negocio del dominio VentaProvisional.
VentaService	Lógica de negocio del dominio Venta.

MovimientoService es el servicio con la responsabilidad explícita de centralizar operaciones del kardex/inventario y el CPP. [R2]

DIAGRAMA PENDIENTE DT-03
 Diagrama de componentes internos
 Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-03. Diagrama de componentes internos. Fuente: código PlantUML verificado contra el proyecto.

9.5. Estructura principal del proyecto

```

app/
├── Http/Controllers/    # Capa HTTP
├── Models/              # Modelos Eloquent
├── Services/            # Lógica de negocio
├── Exports/             # Exportaciones Excel
└── View/Components/    # Componentes Blade

database/
├── migrations/
└── seeders/

resources/
└── views/              # Pantallas Blade por módulo

routes/
└── web.php             # Rutas web principales

tests/
├── Feature/
└── Unit/

```

10. Diseño técnico y tecnologías

10.1. Stack principal

Elemento	Versión / dependencia	Uso
PHP	^8.2	Lenguaje backend.
Laravel Framework	^12.0	Framework web principal.
MySQL	Versión no fijada en repositorio	Persistencia relacional.
Blade	Incluido en Laravel	Renderizado de vistas del servidor.
Tailwind CSS	^4.0.0	Estilos frontend.
Vite	^6.2.4	Compilación y servidor de assets.
Axios	^1.8.2	Solicitudes HTTP/AJAX desde frontend.
Spatie Laravel Permission	^6.20	Roles y permisos.
Yajra DataTables	12.0	Tablas de datos en servidor/cliente.
barryvdh/laravel-dompdf	^3.1	Generación de PDF.
maatwebsite/excel	^3.1	Importación/exportación Excel.
Simple QrCode	^4.2	Generación de códigos QR donde se utilice.
PHPUnit	^11.5.3 (dev)	Pruebas automatizadas.
Laravel Pint	dependencia dev	Formato/estilo PHP.

10.2. Comandos de desarrollo disponibles

```

php artisan serve
npm run dev
composer run dev
php artisan test
./vendor/bin/pint
./vendor/bin/pint --test
php artisan optimize:clear
php artisan migrate
php artisan db:seed
npm run build

```

Los comandos anteriores están documentados en AGENTS.md y composer/package scripts. [R2][R4][R5]

11. Modelo de datos

11.1. Inventario de entidades Eloquent

Dominio	Modelos detectados
Seguridad y configuración	User, Configuracion, AuditoriaEvento, RectificacionHistorial, SunatCertificado, ComprobanteSerie, ComprobanteTipo, DocumentoTipo, AfectacionTipo, Unidad, OperacionTipo, PagoForma, PagoMedio, CobranzaTipo, Linea
Terceros	Cliente, Proveedor
Compras	Compra, CompraDetalle, CompraProvisional, CompraProvisionalDetalle
Ventas y entregas	Venta, VentaDetalle, VentaProvisional, VentaProvisionalDetalle, VentaEntrega, VentaEntregaDetalle, Cotizacion, CotizacionDetalle
Producción / formulación	Formulacion, FormulacionDetalle, Preparada, PreparadaDetalle, Nucleo, NucleoDetalle, NucleoPreparada, NucleoPreparadaDetalle, TipoRacion, Ingrediente, IngredienteDatoNutricional, FormulaAlimento, FormulaAlimentoDetalle, FormulaAlimentoCerdo, FormulaAlimentoCerdoDetalle
Caja, gastos y costos	CajaIngreso, CajaPago, Gasto, GastoCategoria, GastoTipo, Costo, CostoCategoria, CostoTipo
Préstamos	Prestamo, PrestamoDetalle
Planilla	Empleado, EmpleadoSueldo, EmpleadoVacacion, PlanillaAdelanto, PlanillaPrestamo, PlanillaPrestamoPago, PlanillaPago, PlanillaPagoDetalle, PlanillaInasistencia, PlanillaAsistencia

11.2. Entidades críticas y campos conocidos

Usuario (User)

Campo	Descripción
name	Nombre del usuario.
email	Identificador de acceso.
password	Contraseña; el modelo la castea como hashed y la oculta en serialización.
activo	Bandera usada por el proceso de autenticación para permitir/bloquear sesión.
roles/permisos	Relaciones gestionadas por Spatie mediante HasRoles.

Producto

Campo	Descripción
unidad_codigo	Unidad relacionada con el producto.
afectacion_tipo_codigo	Tipo de afectación.
linea_id	Línea de producto.
codigo / nombre / descripcion	Identificación y descripción comercial.
empaque	Factor de empaque usado en conversiones de cantidades.
stock_almacen	Stock operativo; internamente documentado en sacos/empaques.
stock_minimo	Umbral para alertas del dashboard.
costo_unitario	Costo unitario de referencia/valoración.
activo	Estado del producto.

Movimiento (kardex valorizado)

Grupo	Campos principales	Uso
Origen	fecha, tipo, transaccion_tipo, transaccion_id, detalle_id	Identifica fecha y transacción que origina el movimiento.

Grupo	Campos principales	Uso
Producto	producto_id, producto_nombre, empaque, unidad_codigo	Contexto del producto.
Cantidad	cantidad, cantidad_kg, entrada, salida	Magnitud y sentido del movimiento.
Costo	costo_unitario, costo_total	Valoración del movimiento.
Cadena	stock_anterior, costo_actual, valor_anterior, stock_nuevo, costo_nuevo, valor_nuevo	Permite reconstruir el CPP y la valorización antes/después.
Auditoría	user_id, comentario	Responsable/contexto adicional.

Planilla

Modelo / tabla	Campos o reglas documentadas
Empleado / empleados	Nombre, DNI, teléfono, correo, sueldo_planilla, sueldo_real, estado.
PlanillaAdelanto / planilla_adelantos	numero_interno, empleado_id, monto, fecha, observaciones, importes Principal/Depósito/Consortio.
PlanillaPrestamo / planilla_prestamos	numero_interno, empleado_id, monto_original, saldo_pendiente, fecha_prestamo, estado.
PlanillaPrestamoPago / planilla_prestamo_pagos	Pagos parciales del préstamo.
PlanillaPago / planilla_pagos	empleado_id, mes, año, sueldo_base, horas_extras, descuento_faltas, total_pagar.
PlanillaPagoDetalle / planilla_pago_detalles	Detalles adicionales del pago.
PlanillaInasistencia / planilla_inasistencias	empleado_id, fecha, medio_dia, observacion.
PlanillaAsistencia / planilla_asistencias	Asistencias registradas.

DIAGRAMA PENDIENTE DT-04
Diagrama entidad-relación (ER) general
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-04. Diagrama entidad-relación (ER) general. Fuente: código PlantUML verificado contra el proyecto.

11.3. Observación de deuda técnica en unidades

Deuda técnica: AGENTS.md indica que productos.stock_almacen se almacena en SACOS y que nucleo_preparada_detalles.salida_kg, pese al nombre, puede almacenar SACOS. Este desacople entre nombre físico y semántica aumenta el riesgo de errores; se recomienda migrar a nombres explícitos o documentar el contrato de unidad en código, base de datos y pruebas. [R2]

12. Rutas e interfaces funcionales

La aplicación está organizada principalmente mediante rutas web autenticadas. No se observó una especificación OpenAPI/Swagger ni un conjunto independiente de endpoints REST versionados. Por ello, esta sección documenta rutas funcionales, no una API pública. [R3]

Módulo	Prefijo principal	Operaciones observadas
--------	-------------------	------------------------

Módulo	Prefijo principal	Operaciones observadas
Acceso	/login, /logout, /perfil	Inicio/cierre de sesión y perfil.
Dashboard	/dashboard	Indicadores operativos y alertas de stock.
Usuarios/Roles	/usuarios, /roles	Administración de cuentas, roles y permisos.
Productos	/productos	CRUD, detalle y búsquedas.
Clientes	/clientes	CRUD y búsqueda.
Proveedores	/proveedores	CRUD y búsqueda.
Compras	/compras	Registro/consulta/impresión/anulación; sin update/destroy convencional.
Compras provisionales	/compra-provisionales	Operaciones provisionales y saldos.
Ventas	/ventas	Registro/consulta/impresión/anulación/rectificación.
Ventas provisionales	/venta-provisionales	Operaciones provisionales y saldos.
Entregas	/venta-entregas	Pendientes, registro, impresión, anulación y rectificación.
Cotizaciones	/cotizaciones	CRUD e impresión.
Caja	/caja-ingresos, /reportes/caja	Ingresos de caja y reportes general/detallado. El código de /caja-pagos permanece como flujo no expuesto en el menú activo.
Préstamos	/prestamos	Registro, consulta, impresión, anulación y rectificación.
Formulaciones	/formulaciones	Formulaciones y detalles.
Fórmulas alimento	/formulas-alimento	CRUD y exportaciones.
Fórmulas alimento cerdo	/formulas-alimento-cerdo	CRUD y exportaciones.
Preparadas	/preparadas	Registro/consulta/impresión/anulación.
Núcleos	/nucleos	Gestión de núcleos.
Núcleos preparados	/nucleo-preparadas	Registro/consulta/impresión/anulación/rectificación.
Gastos	/gastos	CRUD, impresión y reportes.
Costos	/costos	CRUD, impresión y reportes.
Planilla empleados	/planilla-empleados	Gestión de empleados.
Planilla adelantos	/planilla-adelantos	Adelantos y ticket.
Planilla préstamos	/planilla-prestamos	Préstamos, cuotas/pagos y tickets.
Planilla pagos	/planilla-pagos	Generación, confirmación, reversión y pago.
Inasistencias	/planilla-inasistencias	Registro de inasistencias.
Vacaciones	/empleado-vacaciones	Elegibles, resumen, CRUD y reportes.
Auditoría	/auditoria	Listado y detalle para administrador.

12.1. Familias de reportes

Familia	Contenido
Compras	Detalle por proveedor/fecha, productos destacados y operaciones recientes.
Ventas	Emitidas, por tipo de comprobante, acumuladas, agrupadas, entregas pendientes, clientes, productos y últimas operaciones.
Formulación / producción	Por fechas, rangos y acumulados para formulaciones, preparadas y núcleos preparados.
Préstamos	Pendientes y generales.
Caja	General y detallado, incluido PDF.
Kardex / stock	Stock general, movimientos por rango, Excel y PDF.
Cuenta corriente cliente	Créditos por cobrar, vencimientos/días, saldos, estados de cuenta y rentabilidad.
Cuenta corriente proveedor	Compras, créditos por pagar, estado de cuenta, compras por rango y saldos.
Rentabilidad	Ventas, detalle, productos y productos vendidos.
Planilla	Mensual, empleado, adelantos, préstamos, pendientes, salarios, históricos, inasistencias, trabajadores y ZIP de PDFs.
Gastos / costos	Reportes generales/detallados, exportaciones e impresión.

13. Seguridad

13.1. Controles verificados

- Autenticación de correo y contraseña mediante Laravel Auth. [R9]
- Verificación explícita del estado activo del usuario. [R9]
- Dos capas de limitación de intentos de inicio de sesión: middleware de ruta y RateLimiter en controlador. [R3][R9]
- Contraseña protegida por cast hashed y oculta en serialización del modelo User. [R7]
- Autorización por roles/permisos con Spatie Laravel Permission. [R7][R8]
- Ruta de auditoría restringida a usuario administrador. [R3]
- Convención de uso de transacciones de base de datos para operaciones multi-paso. [R2]

13.2. Hallazgos de seguridad y configuración

ID	Severidad	Hallazgo	Acción recomendada
SEG-01	ALTA	Se observó un archivo .env versionado en el repositorio público.	Retirarlo del índice e historial cuando corresponda, rotar cualquier secreto que haya estado expuesto y agregar .env al .gitignore. Mantener solo una plantilla .env.example sin secretos. [W4][W7]
SEG-02	ALTA	El seeder de roles crea una cuenta administrativa de prueba con credenciales definidas en código.	Eliminar credenciales hardcodeadas de entornos productivos; crear el primer administrador mediante variable segura, comando interactivo o procedimiento de aprovisionamiento. Rotar cualquier credencial existente.
SEG-03	MEDIA	No se encontró .env.example.	Agregar una plantilla documentada sin valores sensibles para hacer reproducible la configuración.
SEG-04	MEDIA	No se encontró pipeline de GitHub Actions en la rama revisada.	Implementar pruebas, Pint, análisis de dependencias y secret scanning en pull requests.
SEG-05	MEDIA	No se observó infraestructura de despliegue versionada (Docker/IaC).	Documentar cPanel, versión de PHP/Apache, variables, permisos de storage, backups y rollback.

13.3. Gestión de secretos

RECOMENDADO - Laravel documenta que el archivo .env no debe confirmarse en el control de versiones porque los valores varían por ambiente y pueden contener información sensible. OWASP también recomienda evitar secretos en repositorios y utilizar mecanismos de gestión/rotación apropiados. [W4][W7]

- Separar configuración de desarrollo, pruebas y producción.
- No registrar APP_KEY, contraseñas de base de datos, claves de correo, tokens o certificados privados en Git.
- Aplicar rotación de secretos después de cualquier exposición.
- Restringir permisos del archivo de entorno en el servidor.
- Usar secretos del proveedor de CI/CD o un gestor de secretos si se automatizan despliegues.

14. Configuración del sistema

El repositorio no contiene una plantilla .env.example. La siguiente tabla es una guía de configuración mínima típica de Laravel y debe ajustarse al ambiente real. No contiene valores reales. [W4]

Variable	Necesidad	Descripción
----------	-----------	-------------

Variable	Necesidad	Descripción
APP_NAME	Sí	Nombre lógico de la aplicación.
APP_ENV	Sí	local / testing / production.
APP_KEY	Sí	Clave de cifrado generada por Laravel; nunca versionar.
APP_DEBUG	Sí	false en producción.
APP_URL	Sí	URL base del sistema.
DB_CONNECTION	Sí	mysql.
DB_HOST	Sí	Host de MySQL.
DB_PORT	Sí	Puerto, normalmente 3306.
DB_DATABASE	Sí	Base de datos; AGENTS.md documenta villegas2026 para desarrollo.
DB_USERNAME	Sí	Usuario de base de datos.
DB_PASSWORD	Sí	Contraseña; secreto.
SESSION_DRIVER	Según entorno	Driver de sesión.
CACHE_STORE	Según entorno	Driver de caché.
QUEUE_CONNECTION	Según uso	Conexión de cola; composer run dev incluye queue:listen.

15. Instalación para desarrollo

Estado: El repositorio documenta comandos de desarrollo, pero no incluye .env.example ni una receta completa de aprovisionamiento. Los pasos siguientes combinan lo verificado con una secuencia estándar recomendada para Laravel 12. [R2][W4]

15.1. Prerrequisitos

- PHP 8.2 o superior y extensiones requeridas por Laravel.
- Composer.
- MySQL.
- Node.js y npm compatibles con Vite 6.
- Git para obtener el código.

15.2. Procedimiento sugerido

1. Clonar el repositorio y ubicarse en la rama aprobada para el ambiente.
2. Ejecutar composer install.
3. Crear un archivo .env local a partir de una plantilla interna segura. Actualmente el repositorio no ofrece .env.example; se recomienda crearlo.
4. Configurar APP_URL, conexión MySQL y demás variables del ambiente.
5. Ejecutar php artisan key:generate.
6. Ejecutar php artisan migrate y los seeders estrictamente necesarios. Revisar el seeder de roles antes de usarlo en producción porque contiene una cuenta de prueba.
7. Ejecutar npm install.
8. Para desarrollo, iniciar php artisan serve y npm run dev, o usar composer run dev.
9. Ejecutar php artisan test antes de validar el ambiente.

```
composer install
php artisan key:generate
php artisan migrate
npm install
composer run dev
php artisan test
```

CAPTURA PENDIENTE DT-05
Entorno de desarrollo ejecutándose
Reemplazar este recuadro con una captura anonimizada del ambiente autorizado.

Figura DT-05. Entorno de desarrollo ejecutándose. Fuente: captura del sistema Consorcios Villegas E.I.R.L.

16. Despliegue a producción

VERIFICADO CON EL RESPONSABLE DEL SISTEMA: producción utiliza cPanel. El monolito Laravel se mantiene como una sola aplicación lógica, pero se separa físicamente: el contenido de public se publica en public_html y el resto del proyecto permanece en una carpeta privada hermana. No se encontraron Dockerfile ni pipeline CI/CD. [W3]

16.1. Arquitectura de producción en cPanel

La carpeta privada contiene app, bootstrap, config, database, resources, routes, storage, vendor, artisan y .env. public_html contiene index.php, .htaccess, build y los demás assets copiados desde public; nunca debe contener .env, app, vendor ni la raíz completa del repositorio.

DIAGRAMA PENDIENTE DT-06
Diagrama de despliegue productivo
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-06. Diagrama de despliegue productivo. Fuente: código PlantUML verificado contra el proyecto.

16.2. Secuencia de release recomendada

10. Crear una versión/tag aprobada del código.
11. Respalda base de datos antes de migraciones con riesgo.
12. Instalar dependencias PHP sin dependencias de desarrollo y optimizar autoloader.
13. Construir assets frontend para producción.
14. Configurar el .env de producción fuera del repositorio y asegurar APP_DEBUG=false.
15. Ejecutar migraciones bajo ventana de mantenimiento si aplica.
16. Ejecutar optimizaciones/cachés de Laravel.
17. Asegurar permisos de storage y bootstrap/cache para el usuario del servidor web.
18. Reiniciar workers/servicios necesarios y ejecutar smoke tests.
19. Conservar procedimiento de rollback y evidencia del release.

```
composer install --no-dev --optimize-autoloader
npm ci
npm run build
php artisan migrate --force
php artisan optimize
```

En cPanel, public_html contiene únicamente el contenido publicable de Laravel. El archivo public_html/index.php carga maintenance.php, vendor/autoload.php y bootstrap/app.php desde la carpeta

privada del backend y ejecuta \$app->usePublicPath(__DIR__) para que public_path() apunte a public_html. Esta adaptación es necesaria porque el proyecto usa public_path() para archivos y reportes. [W3]

17. Flujos de proceso críticos

17.1. Autenticación

Flujo verificado: el usuario envía correo/contraseña; la aplicación valida formato, aplica limitación de intentos, intenta autenticación y finalmente verifica que el usuario esté activo. Si la cuenta está inactiva, la sesión se cierra y se rechaza el acceso. [R9]

DIAGRAMA PENDIENTE DT-07
Diagrama de secuencia de inicio de sesión
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-07. Diagrama de secuencia de inicio de sesión. Fuente: código PlantUML verificado contra el proyecto.

17.2. Venta e impacto en inventario

La venta se registra mediante VentaController/VentaService y la arquitectura del proyecto exige que cualquier salida de inventario sea procesada mediante MovimientoService. La operación debe conservar la trazabilidad del movimiento y, si la fecha es retroactiva, disparar recálculo cronológico del kardex. [R2]

DIAGRAMA PENDIENTE DT-08
Flujo de venta y kardex
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-08. Flujo de venta y kardex. Fuente: código PlantUML verificado contra el proyecto.

17.3. Compra e ingreso a inventario

La compra utiliza CompraService y, para cambios de stock, MovimientoService. Las compras preparadas como transacciones críticas no se editan/eliminan de forma convencional; las correcciones deben seguir los mecanismos previstos por el sistema. [R2]

DIAGRAMA PENDIENTE DT-09
Flujo de compra y actualización de CPP
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-09. Flujo de compra y actualización de CPP. Fuente: código PlantUML verificado contra el proyecto.

17.4. Rectificación y movimientos retroactivos

Cuando la transacción corregida pertenece a una fecha pasada, la cadena debe reconstruirse en orden cronológico fecha, id. Este requisito es crítico porque el orden de inserción no necesariamente coincide con el orden contable. [R2]

DIAGRAMA PENDIENTE DT-10
Flujo de anulación/rectificación retroactiva
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-10. Flujo de anulación/rectificación retroactiva. Fuente: código PlantUML verificado contra el proyecto.

17.5. Planilla mensual

El módulo calcula pagos por periodo, considerando sueldo base, horas extras e inasistencias. Permite generación masiva, confirmación y reversión según rutas disponibles. Los adelantos no se descuentan automáticamente y deben ser considerados por el usuario durante el pago. [R6]

DIAGRAMA PENDIENTE DT-11
Flujo de planilla mensual
Generar desde 02_Diagramas_Tecnicos_Villegas2026.puml.

Figura DT-11. Flujo de planilla mensual. Fuente: código PlantUML verificado contra el proyecto.

18. Reportes y exportaciones

El sistema posee una cobertura amplia de reportes y utiliza dependencias para PDF y Excel. En planilla también existe descarga ZIP de PDFs por empleado. La matriz exacta de formatos depende de cada ruta/controlador. [R3][R4][R6]

Formato	Uso observado / esperado
Pantalla/DataTables	Listados, filtros y consultas operativas.
PDF	Impresiones de transacciones y reportes en múltiples módulos.
Excel	Kardex/stock, formulación y otros reportes que exponen rutas de exportación.
ZIP	Descarga masiva de PDFs de empleados en planilla.
QR	Dependencia Simple QrCode disponible; validar en qué documentos productivos se utiliza antes de describirlo como requisito.

19. Auditoría y trazabilidad

El repositorio contiene AuditoriaEvento, AuditoriaService y RectificacionAuditoriaService, además de rutas de auditoría para administración. Esto evidencia una capa de trazabilidad enfocada en eventos y rectificaciones. [R3][R12]

- La auditoría debe registrar, como mínimo, usuario, operación, entidad afectada y contexto temporal según la implementación disponible.
- Las rectificaciones deben conservar referencia a la operación corregida y al historial correspondiente.
- RECOMENDADO: definir política de retención de logs/auditoría y asegurar que usuarios operativos no puedan alterar estos registros.

20. Pruebas y validación

20.1. Pruebas existentes detectadas

Tipo	Archivo	Enfoque inferido por nombre/contenido
Feature	AuditoriaControllerTest.php	Controlador de auditoría.
Feature	CuentaCorrienteClienteControllerTest.php	Flujos de cuenta corriente de clientes.
Feature	EmpleadoSueldoServiceTest.php	Comportamiento relacionado con sueldos de empleados.
Feature	MovimientoServiceChronologyTest.php	Integridad cronológica del servicio de movimientos/kardex.
Feature	NucleoActivoTest.php	Reglas asociadas a núcleos activos.
Feature	NumericStringOrderTest.php	Ordenamiento de valores numéricos representados como cadenas.
Feature	ReportePlanillaControllerTest.php	Reportes de planilla.
Feature	VentaRectificacionAbonosTest.php	Rectificación de ventas con abonos.
Unit	PagoInicialTest.php	Reglas de pago inicial.
Unit	RectificacionAuditoriaServiceTest.php	Auditoría de rectificaciones.
Manual/Script	verify_kardex_kg_fix.php	Verificación de correcciones de unidades/kardex.
Manual/Script	verify_kardex_retroactivo.php	Verificación de kardex retroactivo.
Manual/Script	verify_kardex_reuso.php	Verificación de reutilización/consistencia del kardex.

Alcance de esta revisión: Se verificó la existencia de las pruebas, pero no se ejecutó el suite completo contra una base de datos/configuración del proyecto. Por ello, el documento no afirma un porcentaje de pruebas aprobadas ni cobertura.

20.2. Estrategia recomendada de QA

- Ejecutar php artisan test en cada pull request y antes de cada release.
- Agregar pruebas para autenticación, permisos y casos de usuario inactivo.
- Mantener tests de invariantes de kardex para entradas, salidas, rectificaciones y fechas retroactivas.
- Probar reportes con volúmenes altos y datos de borde.
- Agregar smoke tests de los módulos críticos luego del despliegue.
- Medir cobertura con foco en Services, no únicamente en porcentaje global.

21. Respaldo y recuperación

RECOMENDADO - no se encontró política formal de backup en el repositorio: MySQL documenta estrategias de copia lógica/física y recuperación, incluido uso de mysqldump y recuperación a un punto en el tiempo con binary logs. El plan real debe definirse según criticidad y volumen. [W6]

21.1. Política mínima sugerida

Elemento	Recomendación base
RPO	Definir pérdida máxima aceptable de datos. Como punto de partida operativo, evaluar copias diarias + binlogs si el negocio exige recuperación más granular.
RTO	Definir tiempo máximo permitido para restaurar servicio.
Base de datos	Backups automáticos, cifrados y almacenados fuera del servidor principal.
Archivos	Respalidar archivos cargados por usuarios si existen fuera de la base de datos.
Verificación	Realizar restauraciones de prueba periódicas; un backup no probado no debe considerarse suficiente.
Retención	Definir retención diaria/semanal/mensual según política interna y regulación aplicable.

```
# Ejemplo lógico; adaptar usuario, host y nombre de base
mysqldump --single-transaction --routines --triggers DB_NAME > backup.sql

# Restauración de prueba
mysql DB_NAME_RESTORE < backup.sql
```

22. Mantenimiento y operaciones

22.1. Comandos de diagnóstico de kardex

```
# Validar cadena completa o producto específico
php artisan kardex:validar
php artisan kardex:validar --producto_id=29

# Recalcular desde una fecha
php artisan kardex:recalcular {producto_id} {fecha}
```

- Pausar temporalmente operaciones que puedan modificar stock durante la corrección.
- No editar directamente stock_almacen como mecanismo de corrección.

22.2. Rutina de mantenimiento sugerida

Frecuencia	Actividad
Diaria	Revisar errores de aplicación, fallos de jobs si se usan colas, backups y alertas de stock crítico.
Semanal	Verificar integridad de kardex en productos con incidencias y revisar usuarios/roles recientes.
Mensual	Ejecutar restauración de prueba, revisar crecimiento de base de datos y dependencias vulnerables.
Por release	Ejecutar tests, Pint, backup previo, migraciones controladas, smoke test y registro de cambios.
Trimestral	Auditar permisos, cuentas inactivas, secretos y documentación operativa.

23. Limitaciones, deuda técnica y oportunidades de mejora

ID	Área	Situación	Mejora
DT-01	Versionado	No se declara versión del software.	Adoptar SemVer y tags/releases.
DT-02	Secretos	Archivo .env presente en el repositorio.	Eliminarlo del control de versiones y rotar secretos.
DT-03	Configuración	No existe .env.example.	Crear plantilla segura y documentada.
DT-04	Seguridad inicial	Seeder con cuenta administrativa de prueba hardcodeda.	Cambiar el aprovisionamiento inicial.
DT-05	CI/CD	No se encontró GitHub Actions.	Automatizar tests, Pint y controles de seguridad.
DT-06	Despliegue	No se encontraron manifiestos de infraestructura/Docker.	Versionar receta de despliegue reproducible.

ID	Área	Situación	Mejora
DT-07	Modelo de unidades	Campo salida_kg con semántica de sacos en un contexto.	Normalizar nomenclatura/migrar y cubrir con tests.
DT-08	Documentación	Documentación interna concentrada en AGENTS.md, PLANILLA.md y un documento de kardex.	Mantener esta documentación junto al release y agregar ADRs para decisiones críticas.
DT-09	API	No se observó contrato OpenAPI.	Si terceros necesitan integración, separar y documentar una API estable.

24. Conclusiones y recomendaciones técnicas

El sistema posee un alcance funcional amplio y una estructura de dominio claramente reconocible. La existencia de una capa de servicios, reglas de inmutabilidad para transacciones críticas y pruebas específicas para el kardex muestra especial atención a la integridad de inventario y rectificaciones.

La prioridad técnica debe situarse en la gestión segura de configuración y credenciales, la formalización de releases y despliegue, y la automatización de pruebas. La documentación también debería mantenerse versionada junto con el software para evitar divergencias entre rutas, pantallas y reglas reales.

- Prioridad 1: retirar y rotar secretos potencialmente expuestos y eliminar credenciales de prueba hardcodeadas.
- Prioridad 2: crear .env.example seguro, versionado del sistema y procedimiento de despliegue/rollback.
- Prioridad 3: incorporar CI con tests, Pint y escaneo de secretos/dependencias.
- Prioridad 4: consolidar la semántica de unidades de inventario y eliminar nombres ambiguos.
- Prioridad 5: mantener diagramas ER, arquitectura y manual de usuario actualizados por release.

Anexo A. Glosario

Término	Definición
CRUD	Create, Read, Update, Delete; operaciones básicas de mantenimiento de datos.
CPP	Costo Promedio Ponderado; método utilizado para valoración móvil del inventario según la documentación interna.
Eloquent	ORM de Laravel utilizado por los modelos para acceder a MySQL.
Kardex	Registro cronológico de entradas, salidas, saldos y valorización de inventario.
Rectificación	Flujo controlado para corregir una transacción manteniendo trazabilidad.
Anulación	Cambio de estado/proceso que invalida una operación sin borrar su historia.
Blade	Motor de plantillas de Laravel.
Vite	Herramienta de construcción de assets frontend.
RBAC	Role-Based Access Control; control de acceso basado en roles.
Seeder	Clase de Laravel usada para cargar datos iniciales en la base de datos.
Migration	Definición versionada de cambios en el esquema de base de datos.
RPO	Recovery Point Objective; pérdida máxima de datos tolerable.
RTO	Recovery Time Objective; tiempo máximo de recuperación del servicio.
CI/CD	Integración continua / entrega o despliegue continuo.

Anexo B. Referencia rápida de comandos

Área	Comando	Propósito
Desarrollo	php artisan serve	Inicia servidor Laravel de desarrollo.
Frontend	npm run dev	Inicia Vite en desarrollo.
Desarrollo	composer run dev	Ejecuta procesos de desarrollo definidos por Composer.
Pruebas	php artisan test	Ejecuta suite de pruebas.
Calidad	./vendor/bin/pint --test	Valida estilo sin modificar archivos.
Calidad	./vendor/bin/pint	Corrige estilo PHP.
Cache	php artisan optimize:clear	Limpia cachés de Laravel.
DB	php artisan migrate	Aplica migraciones.
DB	php artisan db:seed	Ejecuta seeders.
Build	npm run build	Compila assets de producción.
Kardex	php artisan kardex:validar	Valida cadena del kardex.
Kardex	php artisan kardex:recalcular {producto_id} {fecha}	Recalcula desde una fecha.

Anexo C. Patrón de permisos

El seeder genera permisos CRUD con el patrón {tabla}_{acción}, donde acción pertenece a list, create, edit y delete. Además define permisos específicos de reportes y módulos especializados. [R8]

Grupo	Ejemplos verificados
CRUD	productos_list, productos_create, productos_edit, productos_delete; ventas_list, ventas_create, ...
Dashboard/reportes	dashboard_estadisticas, dashboard_productos, ventas_report, compras_report, kardex_report, stock_report, caja_report, rentabilidad_report, planilla_report.
Raciones/fórmulas	ration_datos_*, ration_formulacion_*, formulas_alimento_*, formulas_alimento_cerdo_*.
Especial	super_admin.

Anexo D. Checklist de diagramas e imágenes técnicas

Los diagramas DT-01, DT-02, DT-03, DT-04 y DT-06 a DT-11 pueden generarse con 02_Diagramas_Tecnicos_Villegas2026.puml. DT-05 continúa siendo una captura real del entorno de desarrollo, no un diagrama.

ID	Vista / diagrama	Qué debe mostrar	Archivo sugerido
DT-01	Diagrama de contexto del sistema	Usuario/Administrador -> Sistema Villegas2026 -> MySQL. Como sistemas externos solo agregar los que realmente se usen en el entorno de producción; no inventar integraciones. Sugerencia: diagrama C4 nivel Contexto.	DT-01_diagrama_de_contexto_del_sistema.png
DT-02	Diagrama de arquitectura lógica / contenedores		DT-02_diagrama_de_arquitectura_lógica_contened.png
DT-03	Diagrama de componentes internos		DT-03_diagrama_de_componentes_internos.png
DT-04	Diagrama entidad-relación (ER) general		DT-04_diagrama_entidad_relación_er_general.png
DT-05	Entorno de desarrollo ejecutándose		DT-05_entorno_de_desarrollo_ejecutándose.png
DT-06	Diagrama de despliegue productivo		DT-06_diagrama_de_despliegue_productivo.png
DT-07	Diagrama de secuencia de inicio de sesión		DT-07_diagrama_de_secuencia_de_inicio_de_sesión.png
DT-08	Flujo de venta y kardex		DT-08_flujo_de_venta_y_kardex.png
DT-09	Flujo de compra y actualización de CPP		DT-09_flujo_de_compra_y_actualización_de_cpp.png
DT-10	Flujo de anulación/rectificación retroactiva		DT-10_flujo_de_anulación_rectificación_retroactiva.png
DT-11	Flujo de planilla mensual		DT-11_flujo_de_planilla_mensual.png

Anexo E. Referencias y pendientes de validación

Los siguientes datos dependen del ambiente productivo y deben mantenerse actualizados en el registro operativo de la organización:

- Confirmar versión comercial o número de release del software.
- Confirmar topología real de producción (servidor web, PHP-FPM, MySQL, backups, colas y certificados).
- Generar el diagrama entidad-relación desde la base de datos real y validar nombres/relaciones de todas las tablas.
- Ejecutar la suite de pruebas en un ambiente controlado y registrar fecha, commit, resultado y evidencias; la mera existencia de tests no implica que todos estén aprobados.
- Mantener actualizados los responsables, la política de respaldo, los objetivos RPO/RTO y los datos de soporte.
- Corregir la exposición de configuración sensible: el árbol del repositorio revisado contiene un archivo .env versionado. Debe retirarse del control de versiones y rotarse cualquier secreto que haya quedado expuesto.
- Eliminar o reemplazar las credenciales administrativas de desarrollo definidas en el seeder antes de producción.

Referencias principales

Fuente	Ubicación
Repositorio Villegas2026	https://github.com/JhonMM27/Villegas2026
Laravel 12	https://laravel.com/docs/12.x
ISO/IEC/IEEE 15289:2019	https://www.iso.org/standard/74909.html
C4 Model	https://c4model.com/

APROBACIÓN Y CONTROL DE ENTREGA

El presente documentación técnica y manual del desarrollador - villegas2026 corresponde a la versión 1.1, revisada el 29/08/2026. Su aprobación se formaliza mediante las firmas de los responsables indicados a continuación.

Rol	Responsable	Firma y fecha
Elaborado / actualizado por	Administración / TI	
Revisado por	Responsable funcional	
Aprobado por	Propietario del sistema	

Nota de control: las capturas identificadas como pendientes pueden incorporarse posteriormente sin alterar el contenido funcional aprobado, siempre que correspondan a la misma versión del sistema y no expongan datos personales ni credenciales.